

軟體品質保證 (SQA)

第四章：軟體檢視 (Software Inspection)

授課教師：薛念林教授 (with Gemini AI)

4.1 基本概念

到了測試階段才突然重視起品質，為時已晚。

4.1 軟體檢視與動態測試

- 動態測試的盲點：
 - 只看到「冰山的一角」——動態執行受限於特定輸入路徑與環境狀態。
 - 這次執行通過，未必代表程式邏輯完全正確。
- 軟體檢視 (Software Inspection)：
 - 透過觀看與靜態審查原始碼、規格書、設計圖找出**根本錯誤與異常**。
 - 包含不一致的型態定義、風格差異與架構違規。
- 優點：
 - 單次檢視可發現多個深層錯誤根源（不被其他錯誤遮蔽）。
 - 促進團隊領域知識與程式技術傳承，建立品質文化。



4.1.1 檢視的前置條件

- 明確且正確的規格：有客觀標準作為比對基準。
- 熟悉組織規範：審查成員充分理解團隊標準與 Coding Style。
- 完成特定基準版本：受審產出物已達一定完整度。
- 準備專屬檢核表 (Checklist)：避免遺漏重要檢查維度。
- 管理層支持：
 - 接受檢視初期會增加前置成本（換取後期維護成本大幅下降）。
 - 嚴禁將檢視結果作為員工績效考核的懲罰工具。

Concept Check Question (CCQ 1)

靜態測試（如軟體檢視、規格檢視）可以在程式碼實際執行之前，檢查需求、設計、程式碼甚至測試資料中的異常，以早期發現錯誤、降低整體的軟體品質成本。

- **A.** 正確
- **B.** 錯誤



CCQ 1 - 答案與解析

正確答案：A

- 解析：

- **正確**：靜態測試的主要優勢在於不需執行程式即可找出問題。
- 可應用於軟體開發生命週期的任何階段（包括規格書、設計圖與程式碼）。
- 透過早期發現缺陷，能大幅降低後期修復 Bug 的成本（品質成本）。



4.2 檢視方法與流程

同事找到錯誤，絕對比顧客找到錯誤來得好。

4.2 檢視形式與考量因素

- 形式比較：檢視 (Inspection) vs 走查 (Walkthrough)：

構面 檢視 (Inspection)	走查 (Walkthrough)
形式 正式 (Formal)	非正式 (Informal)
計畫 事先指派角色與分配任務	未事先計畫
導讀 由閱讀者 (Reader) 逐段朗讀	由作者 (Author) 親自講解
記錄 指派專門記錄者 (Scribe)	通常由作者自行記錄
主席 獨立協調者 (Moderator)	無特定主持人

- 關鍵考量：是否事先閱讀、是否需要會前會、是否使用查核表與工具輔助。

4.2.1 檢視標準流程與角色分工

- 標準流程六大階段：

1. 計畫 (**Planning**)：挑選合適成員，指派角色。
2. 概述/會前會 (**Overview**)：作者說明系統背景與規格規則。
3. 個別準備 (**Preparation**)：檢視者獨立閱讀並標記潛在缺陷。
4. 檢視會議 (**Inspection Meeting**)：主持人引導、朗讀討論、記錄缺陷。
5. 重做 (**Rework**)：作者依據缺陷清單修復問題。
6. 追蹤/再檢視 (**Follow-up**)：確認修復品質與決定是否通過。

- 五大核心角色：

- 作者 (**Author**)、檢視者 (**Reviewer**)、閱讀者 (**Reader**)、記錄者 (**Scribe**)、主持人 (**Moderator**)

Software Inspection Process



1. Planning

Organization, schedule and organization.

2. Overview

Initial briefing, Initial about the integration.

3. Individual Preparation

Self-study and review of documents.

4. Inspection Meeting

Group review session in work: review.

5. Rework

Correction of the issue of identified defects, and modifications.

6. Re-inspection

Final verification, verification and quality check.

4.2.4 AI 輔助下的現代檢視實踐

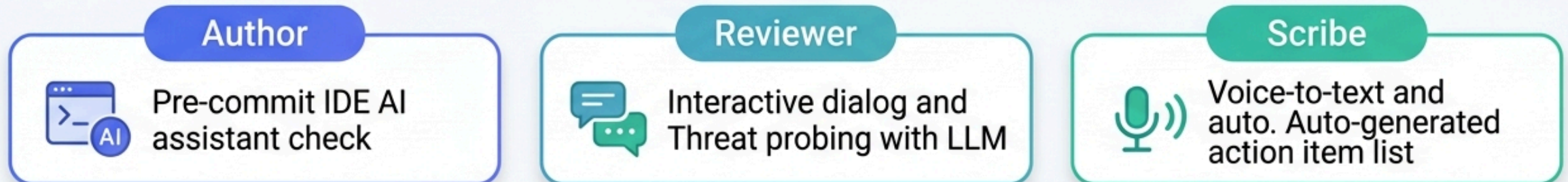
- 流程典範轉移：從「純人工會議」到「AI 先行，人類把關」
 - **Stage 1: AI 第一道防線**：PR 機器人自動掃除低階錯誤、語法規範、基礎資安漏洞並生成 PR 變更摘要。
 - **Stage 2: 人類專家複審**：聚焦於 AI 難以看透的全局架構、領域商業邏輯與系統不變量。
- 檢視角色 AI 賦能：
 - **作者**：本地 IDE (Copilot/Cursor) 提交前自我 Review 與預防修復。
 - **檢視者**：與 LLM 互動對話，針對並發死結與極端邊界做深度推演。
 - **記錄者**：語音轉文字 (STT) 即時記錄結論並自動生成 Action Items。

AI-ASSISTED INSPECTION: MODERN SQA WORKFLOW

Flow of Review



Role Transformation



Concept Check Question (CCQ 2)

在 Fagan 提出的軟體檢視（Inspection）標準流程中，下列哪一個階段的主要目的是由作者向檢視小組說明背景資料與規則，而非進行實際的程式碼除錯？

- **A.** 準備 (Preparation)
- **B.** 概述 (Overview)
- **C.** 檢視會議 (Inspection Meeting)
- **D.** 重做 (Rework)



CCQ 2 - 答案與解析

正確答案：B

- 解析：

- **A 錯誤**：準備階段是參與者各自研讀材料、尋找缺陷的獨立活動。
- **B 正確**：概述（Overview）階段是由作者向小組簡報，說明背景脈絡與規格規則，幫助小組建立共識。
- **C 錯誤**：檢視會議是全體角色聚集，以朗讀和討論方式發掘與記錄缺陷的會議。
- **D 錯誤**：重做階段是作者在會議後修復發現缺陷的階段。



4.3 規格檢視 (Spec Inspection)

規格若錯了，後續矯正所付出的代價呈指數級放大。

4.3 規格檢視的核心品質屬性

-  **Correct (正確性)**：精確反映真實系統建構需求。
-  **Unambiguous (無歧義性)**：每項條款僅有唯一合理解釋。
-  **Complete (完整性)**：涵蓋所有功能與例外邊界。
-  **Verifiable (可驗證性)**：具備可量化且能在有限成本下檢驗的標準。
-  **Consistent (一致性)**：內部條款與前置文件互不衝突。
-  **Traceable (可追溯性)**：前向追溯至組件測試，後向回溯至來源需求。
-  **Design-Independent (設計獨立性)**：專注「做什麼」，不綁死實作細節。
-  **Understandable (易理解性)**：客戶與利害關係人能輕鬆閱讀理解。

KEY QUALITY ATTRIBUTES OF SOFTWARE SPECIFICATIONS



Correct



Unambiguous



Complete



Verifiable



Consistent



Traceable



**Design-
Independent**



Understandable

4.3 規格撰寫實用準則 (7 大原則)

-  **Structured Format (結構組織)**：完整目錄、章節標題、索引與縮寫定義表。
-  **Visual Diagrams (圖文並茂)**：架構圖輔以文字說明，符號與專有名詞一致。
-  **Concrete Examples (具體範例)**：公式與演算法附帶文字解說與實例數據。
-  **Precise Wording (精確用詞)**：杜絕模糊量詞（「通常、等等」），清楚界定流程。
-  **Realistic Scope (務實邊界)**：避免使用過度絕對詞（「總是、絕不」）。
-  **Active Voice (主動語態)**：避免模糊被動句，明確指名行為主體。
-  **Explicit Versions (明確版本)**：第三方相依套件與協定標註確切版本號。

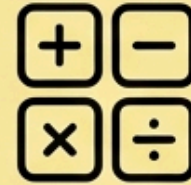
SPECIFICATION WRITING GUIDELINES



**Structured
Format**



**Visual
Diagrams**



**Concrete
Examples**



**Precise
Wording**



**Realistic
Scope**



**Active
Voice**



**Explicit
Versions**

Concept Check Question (CCQ 3)

為了在需求與系統規格階段做到「雙向追溯 (Bidirectional Traceability)」，規格書應該確保具備下列何種關係特性？

- **A.** 每個使用者需求均可對應到特定系統規格，且每個系統規格皆能回溯到其來源需求
- **B.** 規格書的字數與最終程式碼行數必須成固定正比關係
- **C.** 每一行程式碼都必須直接對應到 UML 類別圖的所有屬性
- **D.** 規格書必須僅由開發人員撰寫，完全不允許顧客檢閱



CCQ 3 - 答案與解析

正確答案：A

- 解析：

- **A 正確**：雙向追溯（Bidirectional Traceability）包含前向追溯（需求對應到規格與測試）與後向追溯（規格、程式碼與測試能回溯至起源需求）。
- **B 錯誤**：規格書字數與程式碼行數並無固定正比關係。
- **C 錯誤**：規格追溯並非逐行對應程式碼與 UML 屬性。
- **D 錯誤**：規格檢視必須有顧客或領域專家參與驗證。

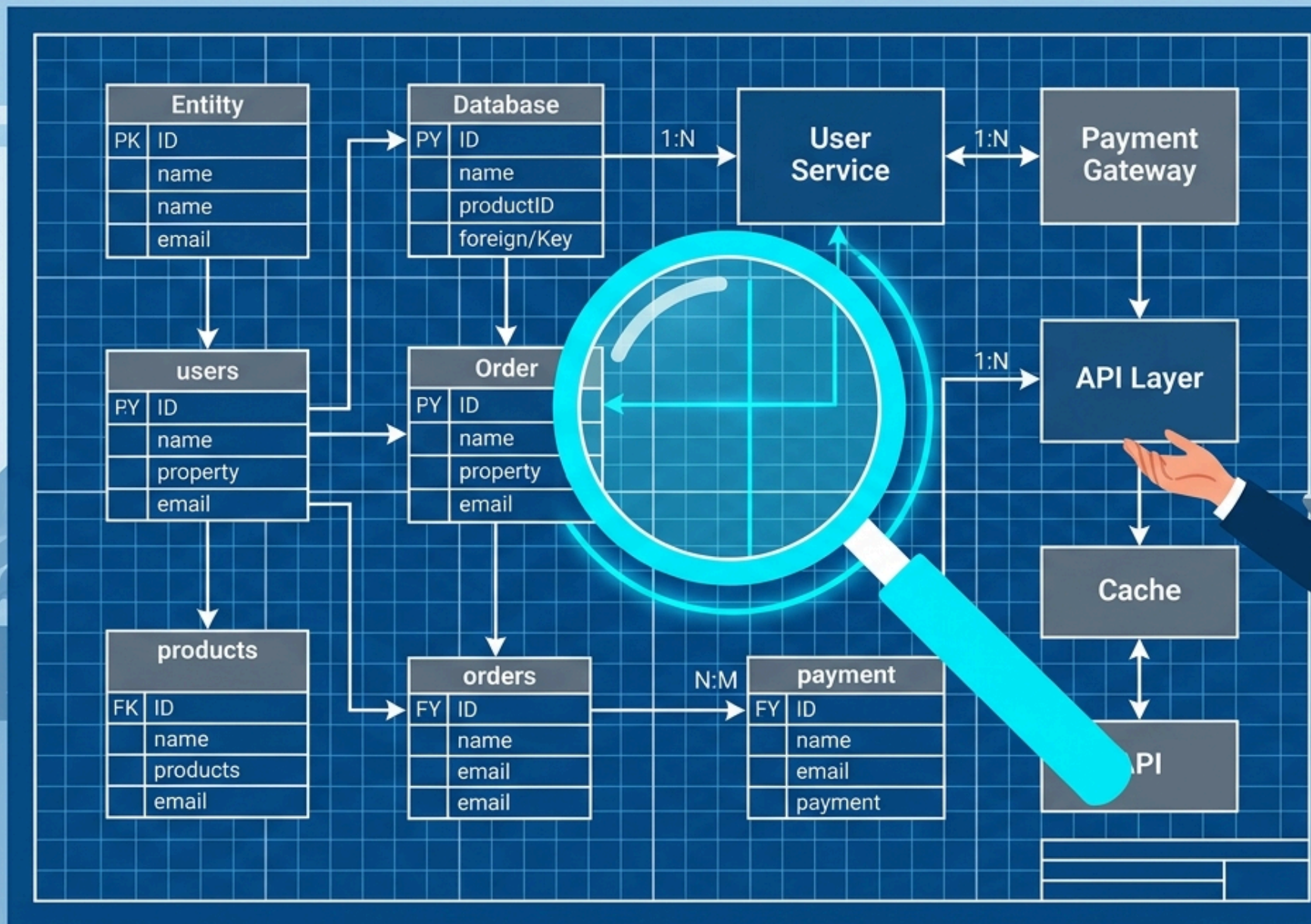


4.4 設計檢視 (Design Review)

及早防錯 (Shift-Left)：在編譯與實作前掃除架構缺陷。

4.4 設計檢視與五大檢核維度

- 核心目的：架構選型防錯、模組解耦、資料庫 Schema 審查與非功能屬性確保。
- 五大設計檢核維度：
 1. 實體與介面完整性 (**Entities & Interfaces**)：識別碼、目的、介面參數與回傳值定義完整。
 2. 架構品質與設計原則 (**Architecture & Principles**)：高內聚、低耦合、模組階層化。
 3. 需求追溯與功能完整 (**Traceability & Completeness**)：架構滿足所有需求，方案具可行性。
 4. 多維度架構視角 (**Architectural Views**)：邏輯視角、行程視角、實體與開發視角。
 5. 關鍵非功能設計議題 (**Key Design Issues**)：例外處理、資源管理、安全與國際化。



4.4.3 設計模型檢核：DFD 核心要素與要點

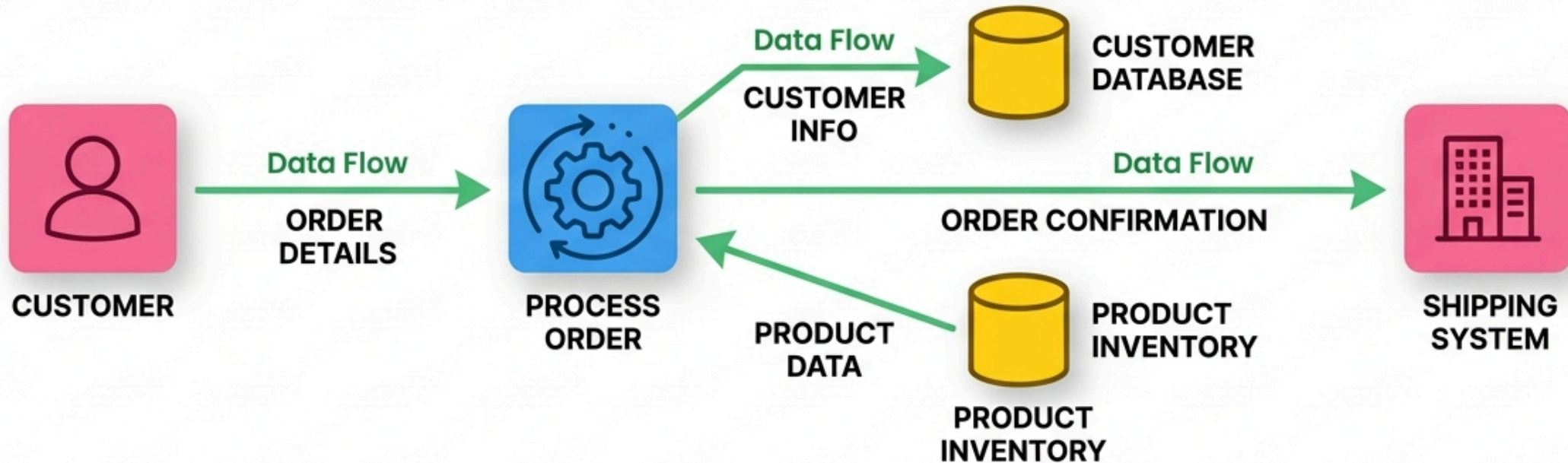
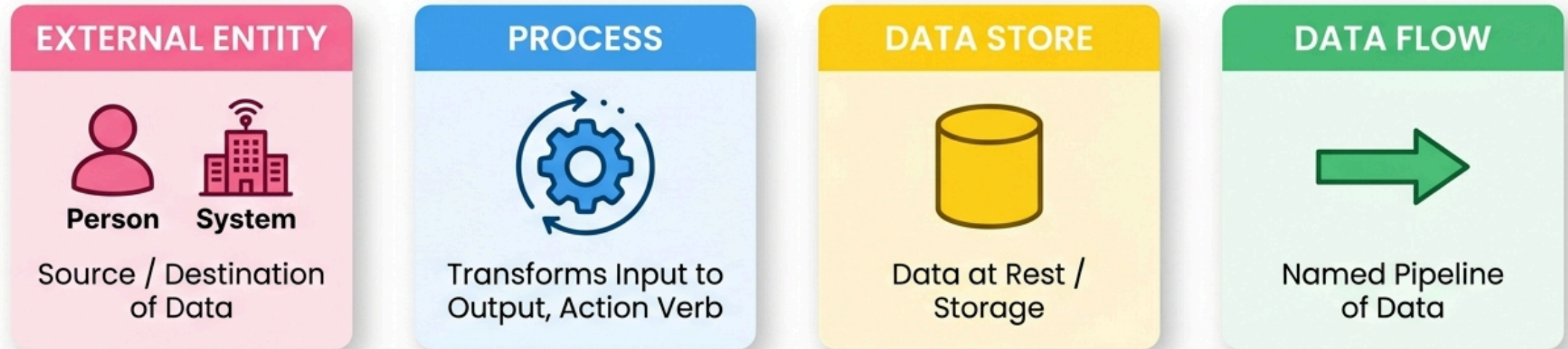
- 四大核心要素：

- 外部實體 (**External Entity**)：資料來源或終點（人或外部系統）。
- 處理過程 (**Process**)：輸入轉換為輸出的動作（動詞 + 名詞命名）。
- 資料儲存 (**Data Store**)：靜態保存資料的儲存庫或表格。
- 資料流 (**Data Flow**)：帶有名稱的有向資料傳遞管道。

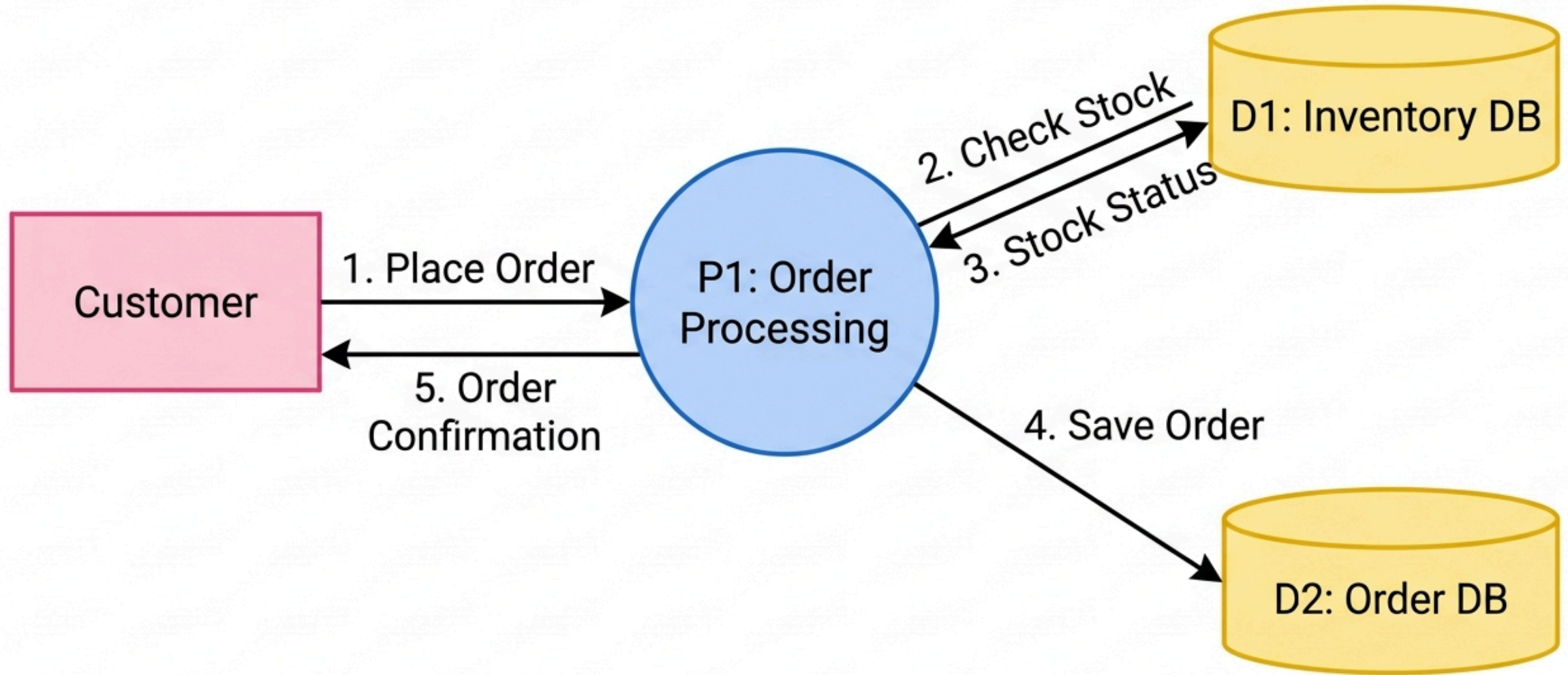
- 檢核法則：

- 每個處理過程（Process）至少有一個輸入和一個輸出（防黑洞/奇蹟）。
- 每個資料儲存至少有一個寫入（輸入）與一個讀取（輸出）。
- 父子圖階層一致性（Context Diagram 與 Level-1 DFD 吻合）。

DATA FLOW DIAGRAM (DFD)



Order Processing System



Concept Check Question (CCQ 4)

設計檢視（**Design Review**）最理想的執行時機，是在系統所有模組的單元測試與整合測試皆通過之後，以確保實際產出的系統與設計文件相符。

- **A.** 正確
- **B.** 錯誤



CCQ 4 - 答案與解析

正確答案：B

- 解析：
 - 錯誤：設計檢視應遵循「及早測試 (Shift-Left)」原則，在程式碼撰寫 (Coding) 開始之前就進行。
 - 如果等到測試階段才發現架構設計的瑕疵，此時資料庫與程式碼都已成形，修改的代價將會非常高昂。



4.5 程式碼檢視 (Code Inspection)

好的程式本身就是最好的註解。

4.5 程式碼檢視常見缺陷分類

-  **Data Faults (資料錯誤)**：變數未初始化即使用、宣告未用、重複賦值未讀取、陣列越界。
-  **Control Faults (控制流程錯誤)**：無法到達的死碼 (Unreachable code)、潛在無窮迴圈。
-  **I/O Faults (輸入/輸出錯誤)**：變數重複輸出未改變、檔案或串流未正確關閉。
-  **Interface Faults (介面錯誤)**：參數型態/數量不吻合、忽略回傳值、未被呼叫的孤立方法。
-  **Coding Standards (編碼規範)**：命名規範、架構分層規範、安全編碼防範。

CODE INSPECTION CATEGORIES & FAULT TYPES



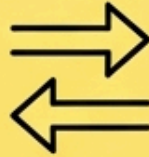
Data Faults

- Uninitialized variables
- Unused variables
- Array out of bounds



Control Faults

- Unreachable dead code
- Infinite loops
- Branching logic error



I/O Faults

- Double output without change
- Resource leaks
- Format mismatch



Interface Faults

- Parameter mismatch
- Ignored return value
- Uncalled functions

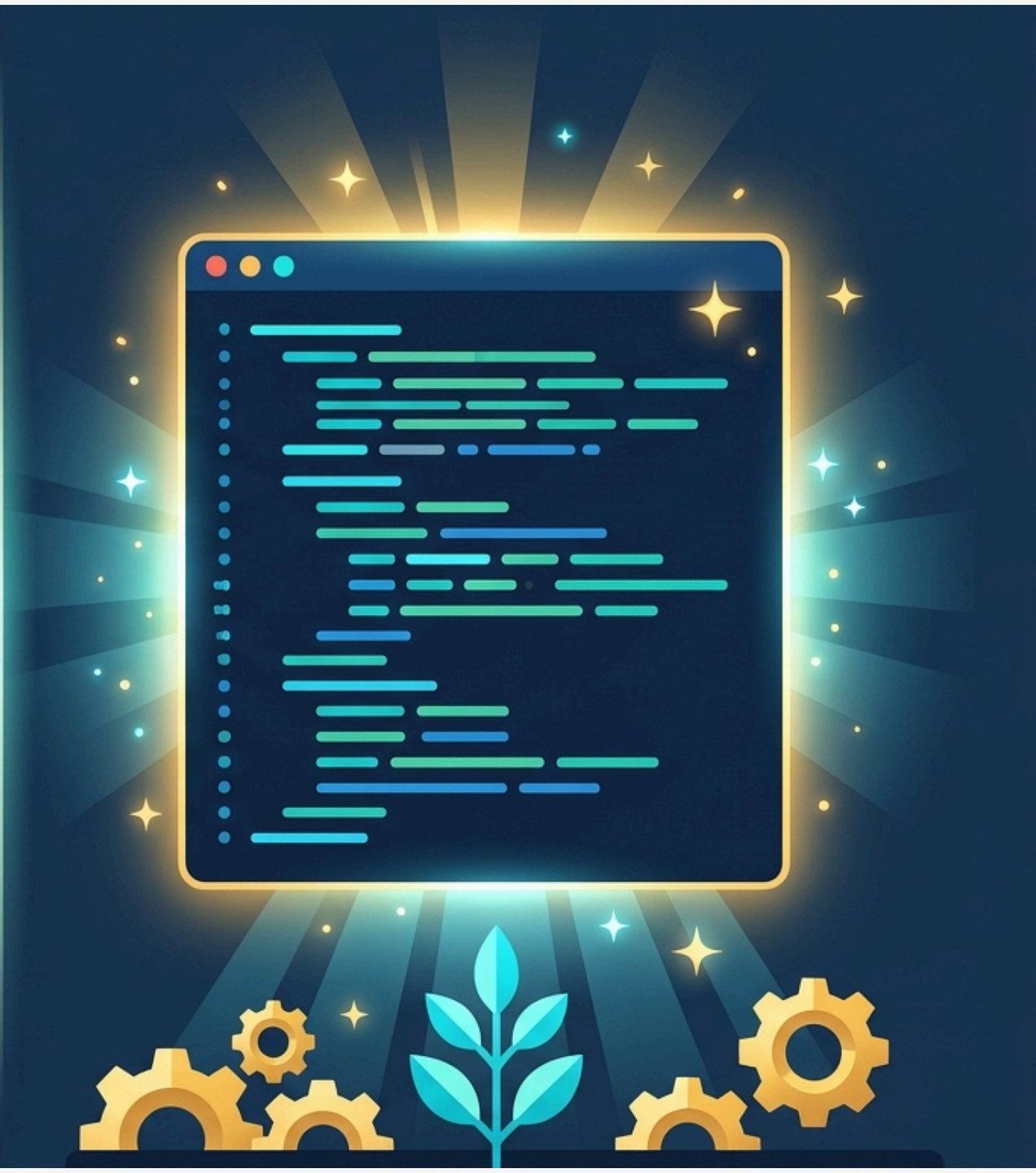


Coding Standards

- Naming rules
- Security coding
- Architecture compliance

4.5.1 程式臭味檢視 (Code Smells)

- 常見 Fowler 程式臭味：
 - **Duplicated Code (重複程式碼)**：抽離至共用方法/元件。
 - **Long Method (冗長方法)**：責任過重，需進行邏輯抽象提煉。
 - **Large Class (大類別)**：職責過多，違反單一職責原則 (SRP)。
 - **Long Parameter List (太長參數列)**：應彙整封裝為參數物件。
 - **Divergent Change & Shotgun Surgery (發散變更與散彈槍手術)**：內聚力低、耦合度高。
 - **Feature Envy (依戀情結)**：方法過度存取其他類別的資料。
 - **Primitive Obsession (基本型別偏執)**：堅持用基礎型態代表具業務語意的實體。



4.5.2 安全漏洞檢視 (OWASP Top 10)

- **A01 權限控制失效 (Broken Access Control)**：未授權存取敏感資源。
- **A02 加密機制失效 (Cryptographic Failures)**：傳輸未加密或使用過時演算法。
- **A03 注入攻擊 (Injection)**：SQL Injection, XSS, OS Command Injection。
- **A04 不安全設計 (Insecure Design)**：缺乏架構級防威脅建模。
- **A05 安全設定錯誤 (Security Misconfiguration)**：洩漏詳細 Exception 堆疊。
- **A06 易受攻擊與過時元件 (Vulnerable Components)**：相依套件含已知漏洞。
- **A07 識別與身分驗證失效 (Auth Failures)**：密碼過弱或 Session 漏洞。
- **A08 軟體與資料完整性失效 (Integrity Failures)**：反序列化不受信任資料。
- **A09 安全記錄與監控失效 (Logging Failures)**：資安事件未記錄與警報。
- **A10 伺服器端請求偽造 (SSRF)**：未驗證 URL 導致內部網絡探測。



Concept Check Question (CCQ 5)

在程式碼檢視中，若發現系統直接將詳細的例外錯誤堆疊資訊（如 `e.printStackTrace()`）輸出至前端頁面或公開日誌，這屬於 OWASP Top 10 中的「A05:2021-安全設定錯誤 (Security Misconfiguration)」漏洞範疇。

- A. 正確
- B. 錯誤



CCQ 5 - 答案與解析

正確答案：A

- 解析：

- **正確**：在生產環境中輸出詳細的 Debug 資訊或錯誤堆疊（Stack Trace），會將系統內部的元件版本、程式碼路徑及資料庫結構暴露給外部。
- 這屬於典型且嚴重的「安全設定錯誤 (Security Misconfiguration)」。
- 正確做法應使用結構化日誌記錄內部，前端僅回傳友善且模糊的錯誤訊息。



4.6 檢視的度量與評估

無法度量的事物，就無法有效改善。

4.6 檢視效益的量化度量指標

- **缺失偵測效率 (Defect Detection Efficiency) :**
 - 每小時檢視發現缺失數、每千行程式碼 (KLOC) 發現缺失數。
- **成本效用 (Cost Effectiveness) :**
 - 審查找出錯誤成本 vs 留到後期測試/上線除錯成本之比率。
- **錯誤密度 (Defect Density) :**
 - 發現缺陷總數 / 軟體規模大小 (LOC 或 FP)。
- **錯誤移除率 (Defect Removal Leverage, DRL) :**
 - 比較不同階段每小時移除錯誤之相對效果，評估流程改善效益。
- **檢視速率建議：**維持在 200–400 LOC/Hr，避免過快導致漏檢。

Concept Check Question (CCQ 6)

組織在推行軟體檢視與審查時，常會使用度量指標來評估其效率。下列關於「檢視速率 (Review Rate)」與「檢視品質」的敘述，何者最為正確？

- **A.** 檢視速率愈快（如每小時檢視 2000 行），代表檢視品質愈高、找出的缺陷愈多
- **B.** 檢視速率過快通常會導致缺陷遺漏率增高，因此應維持在建議的合理速率內
- **C.** 為了大幅提升開發速度，檢視會議應儘可能限制在 5 分鐘內結束
- **D.** 度量指標在軟體工程中的主要目的是用於懲罰寫出最多缺陷的工程師



CCQ 6 - 答案與解析

正確答案：B

- 解析：

- **A 錯誤**：檢視速率太快代表走馬看花，通常會漏掉深層錯誤，品質反而下降。
- **B 正確**：超出合理負荷會導致缺陷漏到後續階段（Defect Leakage），維持合理速率（如 200–400 LOC/Hr）至關重要。
- **C 錯誤**：檢視會議需要充分時間研討與發掘缺陷，5 分鐘無法達成目的。
- **D 錯誤**：度量指標用於過程改善與品質預測，絕非用來懲罰工程師。



本章重點回顧

本章小結與重點

- **靜態檢視的價值**：及早發現深層缺陷、降低總品質成本、傳承團隊知識。
- **標準檢視流程與角色**：六大階段嚴謹執行，明確分工（主持人、作者、閱讀者、記錄者）。
- **AI 現代賦能**：AI 第一道防線掃除低階缺陷，人類專家聚焦架構與商業邏輯。
- **跨階段檢視**：
 - **SRS 檢視**：掌握 8 大品質屬性與 7 大撰寫準則。
 - **設計檢視**：掌握 5 大設計維度與 DFD 檢核要點。
 - **程式碼檢視**：常見缺陷分類、Code Smells 識別與 OWASP Top 10 安全漏洞防護。
- **度量與持續改善**：依據 DRL 與錯誤密度持續優化檢視流程。

Q & A

謝謝大家！